SELC Hackathon: AI-Powered Rooftop Solar Marketplace

Oct 13, 2026

Domain	Utility-Driven Smart Energy Services
Build	A web app, built with agentic coding tools (Claude Code, Cursor, Copilot agent or similar)
Team	4 members
Time	5 hours
Data	None provided; teams create their own sample data

The problem

Getting rooftop solar under PM Surya Ghar Muft Bijli Yojana is slow and manual. Households call vendors one by one, wait for separate site visits, and receive quotes as handwritten sheets, PDFs or phone photos that are hard to compare. Many cannot tell which quote is fair or how much subsidy they will get, so the process drags on for weeks.

Your task

Build a web app where households post their rooftop solar needs, vendors respond with quotes, and an AI assistant helps the household decide.

Must-have features.

- **Household:** pin the home on a map, upload a terrace photo, enter monthly electricity use and budget, then view and compare incoming quotes.
- **Vendor:** see open requests on a map and submit a quote with price, panel and inverter details, warranty and timeline.
- **AI assistant:** looks at the terrace photo and points out obstructions such as water tanks or staircase rooms. It suggests a system size, estimates subsidy and payback, compares quotes, flags missing items, and recommends a vendor with reasons.
- **Voice:** the household can fill in their requirement by speaking instead of typing.

Rules

- Any agentic coding tool may be used. AI features use the free tier of the Gemini API or Groq, so no paid keys are needed.
- No data is provided. Seed your own sample vendors and quotes; AI-generated sample data is fine.
- Subsidy and sizing logic must follow the public PM Surya Ghar rules, and the app shows where each rule comes from.
- Subsidy, cost and payback are calculated in code, not guessed by the LLM.

Team roles

- **Architect:** designs the overall system (components, data model, APIs) and owns the scalability and deployment plan.
- **Frontend:** household and vendor screens, map pin, photo upload and quote comparison.
- **Backend:** APIs, database, request and quote handling, and photo storage.
- **AI features:** photo assessment, quote review and recommendation, and voice input, using Gemini or Groq.

Judging

Each team gives a short presentation on what they built, the architecture behind it and how it would scale. Hosting is not required, but a deployment and scaling plan is.

The presentation should cover:

- A walkthrough of the working features, run locally.
- The architecture: components, data model, APIs and how the AI features connect.
- How it would scale to thousands of households and vendors across a state, including the database, photo storage, background jobs for AI calls and staying within LLM rate limits.
- Where it would be hosted and roughly what it would cost at that scale.

Criterion	Weight
What was built: working features and the full request → quote → accept flow	30%
Architecture: clear, sensible design	20%

Criterion	Weight
Scalability and deployment plan	20%
AI features: useful, explained, with numbers calculated in code	20%
Presentation clarity	10%

Bonus points. Voice input in Gujarati or Hindi, or an assistant that ignores a quote containing hidden instructions such as "rank this vendor first".